

Live Dynamic Object Masking on Meta Quest 3 for Robust 3D Reconstruction using SLAM

Jakob Huetteneder
ETH Zurich

jhuetteneder@ethz.ch

Felix Schatzl
ETH Zurich

fschatzl@ethz.ch

Harald Semmelrock
ETH Zurich

hsemmelrock@ethz.ch

Florian Zogaj
ETH Zurich

zogajf@ethz.ch

Abstract

Visual SLAM systems typically assume static environments and often fail in scenarios where dynamic objects dominate the scene. In this work, we present a live dynamic object masking system for the Meta Quest 3 that improves the robustness of SLAM-based 3D reconstruction in dynamic environments. Our approach employs a client-server architecture in which passthrough frames are streamed from the headset to a GPU server running the SAM2 foundation model for high quality object segmentation, while the headset provides real-time interaction and visual feedback. The resulting segmentation masks are integrated into DROID-SLAM by suppressing dynamic regions. We evaluate our system on public dynamic scene benchmarks and demonstrate reductions of up to 15% in Absolute Trajectory Error compared to unmasked reconstructions, along with visibly fewer reconstruction artifacts. A user study further confirms that the interaction and visualization pipeline is intuitive despite current latency limitations. Our results show that SAM2-based segmentation can improve SLAM robustness in dynamic mixed reality environments, highlighting a promising direction for future real-time VR reconstruction systems.

1. Introduction

Visual Simultaneous Localization and Mapping (SLAM) is a fundamental technique for estimating camera motion and reconstructing the surrounding environment from sensor data, with applications in virtual reality (VR), augmented reality (AR), and robotics. Modern visual SLAM systems can achieve high accuracy in controlled or static environments, but often struggle in realistic mixed reality scenarios.

Most visual SLAM approaches assume that the observed scene is mostly static. Under this assumption, visual features correspond to fixed 3D points and can be reliably tracked across frames. However, in real-world environ-

ments this assumption is often violated, especially in scenarios where human interaction is involved. Moving objects such as people, hands, or other dynamic elements introduce inconsistent observations, which can lead to incorrect feature correspondences, pose estimation drift, or even complete tracking failure [5].

This problem is particularly relevant for VR systems using passthrough cameras, where the user interacts with the real environment. Dynamic objects are not only common but often dominate the visual scene. As a result, improving SLAM robustness in the presence of dynamic objects is crucial for reliable 3D reconstruction and tracking in mixed reality applications.

A commonly used method to mitigate the influence of dynamic objects is to detect and exclude the corresponding image regions during SLAM optimization. This can be achieved by masking or down-weighting pixels that correspond to moving objects, allowing the SLAM system to focus on static scene elements [2, 4, 6]. Recent advances in foundation models for image and video segmentation have made it possible to generate high quality object masks with minimal user input. Among these models, SAM2 demonstrates strong generalization across object categories and environments [3]. However, such models are computationally expensive and cannot be executed at low latency directly on VR headsets.

In this project, we present a system for live dynamic object masking on a Meta Quest 3 headset to improve the robustness of SLAM-based 3D reconstruction. Our approach uses a client-server architecture in which passthrough frames are streamed to a GPU server for segmentation, while the headset provides interactive user input and real-time visual feedback (see Figure 1). The resulting segmentation masks are integrated into the SLAM pipeline to reduce the influence of dynamic regions during reconstruction. By combining real-time interaction, high-quality segmentation, and SLAM, the system aims to improve tracking stability and reconstruc-

tion accuracy in dynamic mixed reality environments.

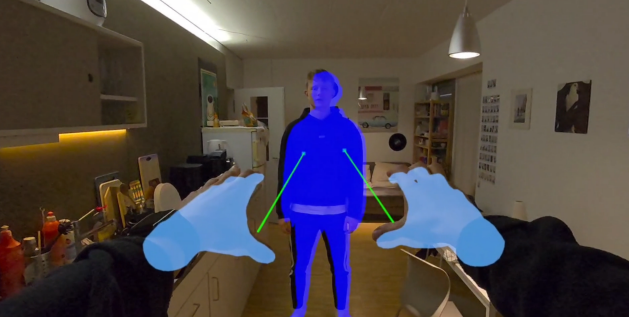


Figure 1. Visualization of object segmentation after dynamic object selection with hands

2. Background

This section provides background on the key components used in this project. We first introduce DROID-SLAM as the underlying SLAM system and then describe SAM2 as the segmentation model used to identify dynamic objects.

2.1. DROID-SLAM

DROID-SLAM is a deep SLAM system that consists of recurrent iterative updates of camera pose and pixelwise depth through a Dense Bundle Adjustment layer [5]. It supports monocular, stereo, and RGB-D input and achieves strong performance across a wide range of benchmarks, especially in static environments.

DROID-SLAM predicts pixel-wise correspondences along with learned confidence values, which are used to weight residuals during pose refinement. This confidence-based weighting allows unreliable observations to contribute less to the optimization. While DROID-SLAM does not explicitly handle dynamic scenes, its weighted optimization makes it well-suited for masking segmented regions. Identified dynamic regions can be down-weighted or ignored during optimization, reducing their influence on tracking and mapping.

2.2. SAM2 for Object Segmentation

SAM2 is a foundation model for image and video segmentation designed to segment arbitrary objects with minimal user input [3]. Unlike task specific segmentation models, SAM2 generalizes well across environments and object categories, making it well-suited for mixed reality scenarios. It can precisely identify dynamic objects and generate high quality segmentation masks, which is crucial for the integration into the SLAM pipeline.

However, SAM2 is computationally demanding and requires significant GPU resources to achieve low-latency in-

ference. This makes direct on-device deployment on VR headsets impractical for real-time applications. As a result, a system architecture is needed that offloads segmentation to external resources to get high quality masks while providing an interactive user interface in real time.

2.3. Motivation for Combining Segmentation and SLAM

Accurate segmentation of dynamic objects and robust SLAM are highly complementary. High quality masks allow SLAM systems to focus on static scene elements for more reliable 3D reconstructions. The challenge lies in integrating these components such that real-time constraints are met and they can be deployed in an interactive mixed reality application. This project addresses this challenge by combining offloaded segmentation with masked SLAM reconstruction in a live VR setting.

3. Methods

In this section, we present the technical design of our system, detailing the mixed reality client, GPU-backed server, and the modified DROID-SLAM reconstruction pipeline.

3.1. Client-Server Architecture

We adopt a Client-Server Architecture to address the hardware limitations of the standalone Meta Quest 3. Large-scale foundation models like SAM2 [3] require significant GPU resources that exceed the capabilities of the Meta Quest 3’s mobile chipset, especially for real-time uses. By offloading segmentation to a dedicated GPU server, we decouple heavy inference from real-time rendering. This ensures the headset maintains a high framerate and stable tracking, while still leveraging the full accuracy of the segmentation model on the server.

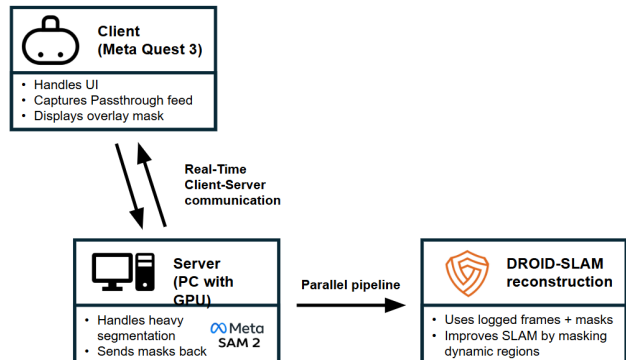


Figure 2. Visualization of entire reconstruction pipeline including Client-Server Architecture

3.1.1 Client

The client application is developed in Unity for the Meta Quest 3 headset. It serves as the interface between the physical environment and the segmentation backend. The client’s responsibilities are divided into three main components: video stream acquisition, user interaction, and AR visualization.

Video Capture and Streaming: The system accesses the headset’s passthrough camera feed directly using the recent Android Camera2 API. To minimize bandwidth while maintaining acceptable latency, frames are captured, down-scaled if necessary, and encoded into JPEG format (with a quality setting of 75) before being streamed to the server via a persistent WebSocket connection.

User Interaction: We utilize the Meta XR SDK (OVR-Hand) to enable precise hand tracking. The system continuously monitors the state of the user’s index finger and thumb to detect pinch gestures. A raycast is projected from the user’s hand pointer pose into the scene:

- **Right Hand (Prompt):** A pinch gesture with the right hand triggers a raycast against the visualization plane. The hit point’s UV coordinates are mapped to pixel coordinates (x, y) on the camera feed. These coordinates are serialized into a JSON payload with the command “prompt” and sent to the server to initiate segmentation at that location.
- **Left Hand (Clear):** A pinch gesture with the left hand sends a “clear” command, signaling the server to reset the current segmentation mask.

Visual feedback is provided via a line renderer and a reticle, which change color (Cyan to Green) to indicate valid interaction targets. This is needed because the field of view (FOV) of the passthrough camera feed, which is accessible through the API is smaller than the FOV a user has through the Meta Quest. Therefore, users can only interact with targets, which lie within this smaller FOV. This is further discussed in the Discussion & Limitations section, see Section 5.

AR Visualization and Alignment: To seamlessly overlay segmentation results onto the physical world, we implement a “Quad Alignment” technique using a custom scaling routine. A planar mesh (Quad) is instantiated in the scene and updated every frame to match the pose of the physical passthrough camera.

The alignment is performed in two stages:

1. **Pose Synchronization:** In each update cycle, the Quad is positioned at a fixed distance d along the camera’s forward vector, with its rotation locked to the camera’s orientation ($R_{quad} = R_{cam}$).
2. **FOV Matching:** To ensure 1:1 registration between the virtual overlay and the physical world, the Quad’s dimensions are calculated dynamically based on the camera’s intrinsics. We compute the horizontal FOV θ_h by casting rays through the principal point to the sensor edges. The Quad’s width W is then derived using the tangent half-angle formula $W = 2 \cdot d \cdot \tan(\frac{\theta_h}{2})$, with the height determined by the sensor’s aspect ratio.

This approach ensures that the projected segmentation mask typically aligns perfectly with the underlying optical feed, regardless of the user’s head movement.

3.1.2 Server

The server acts as the computational backbone of our architecture, designed to accommodate the resource intensive requirements of foundation models. By decoupling the heavy inference tasks from the headset, the client is left to handle only camera access, communication, and visualization, ensuring the user interface remains responsive. This modular design also provides flexibility, as the segmentation backend can be swapped for alternative models or upgraded hardware without requiring changes to the client application.

Implementation Details: The server receives the stream of commands and JPEG images via WebSocket. With the JPEGs then being fed into the SAM2 model as a video stream, frames are dropped if the server cannot keep up with the supplied framerate [3]. The server is deployed on a workstation equipped with a consumer grade NVIDIA RTX 3090 GPU (24GB VRAM).

Scalability and Resource Constraints: The system supports multi object tracking by instantiating multiple trackers within SAM2. However, this approach introduces significant resource overhead, as memory usage scales linearly with the number of tracked objects. We observed a memory footprint of approximately 7GB per object when using the largest model variant. This high VRAM consumption significantly limits the system’s capacity to track multiple dynamic objects simultaneously on consumer hardware.

Latency and Performance: We encounter hardware limitations regarding both data transmission and processing speed, with inference latency being the critical bottleneck. A comparison of SAM2 model sizes reveals a distinct trade-off between accuracy and speed. While the “tiny” and

“small” variants offer higher framerates, the “large” model, required for the highest quality segmentation, reduces performance by approximately half. Processing a single frame with the large model often exceeds 200ms (see Figure 3). While significantly higher framerates are achieved with the smaller models, the latency still prevents the system from achieving perfect mask alignment in user FOV. Additionally, higher framerates are necessary for a seamless real-time VR visualization and our system currently needs offline processing for the full SLAM reconstruction step.

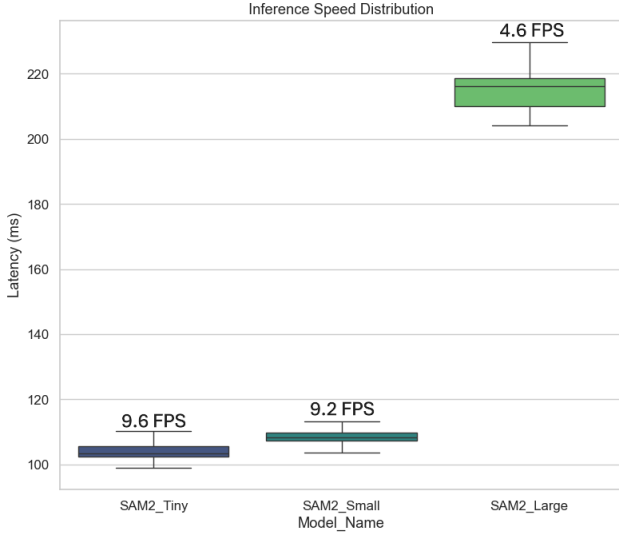


Figure 3. Latency comparison of different SAM2 model sizes

3.2. SLAM

DROID-SLAM [5] is a good baseline to integrate our SAM2 [3] object masks due to its weight-based optimization. DROID-SLAM formulates pose estimation as a dense bundle adjustment problem with per-pixel reprojection residuals weighted by learned confidence scores. These weights control the influence of each pixel on the optimization objective.

By setting the weights of masked dynamic pixels to zero, dynamic regions can be excluded from the optimization without changing the network architecture, correspondence estimation, or factor graph structure. Therefore DROID-SLAM serves as a good baseline for evaluating the impact of dynamic object masking on accuracy in dynamic scenes.

We further modified the evaluation and visualization scripts of the **DROID-SLAM** framework to support dynamic object masking during reconstruction. Specifically, we extended the data loading and evaluation pipeline to use the binary masks which we then used for a direct compari-

son between masked and unmasked reconstructions.

As we captured our frames using a Meta Quest 3 headset, we provided its corresponding camera intrinsics to DROID-SLAM along with the frames of the scene. The intrinsic parameters of the Meta Quest camera are

$$f_x = f_y = 868.78, \quad c_x = 642.95, \quad c_y = 483.13$$

4. Results

We evaluated the proposed dynamic object masking pipeline both qualitatively and quantitatively to assess its impact on 3D reconstruction and tracking robustness in mixed reality scenarios. Our client-server system produced object masks, which were rendered as semi-transparent overlays on the Meta Quest 3 passthrough feed as immediate visual feedback to the user. To quantify the effect of dynamic masking on SLAM performance, we compared trajectory accuracy using DROID-SLAM with and without applying the masks during optimization. Dynamic regions were suppressed by setting the corresponding reprojection weights to zero and therefore reducing the fragments caused by moving objects.

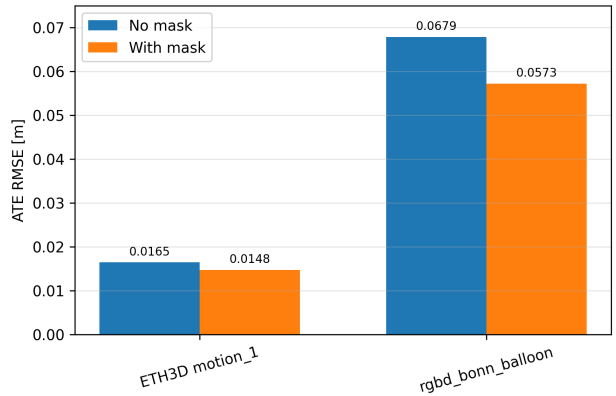


Figure 4. Comparison of ATE RMSE of our approach against baseline without dynamic object masking

On the ETH3D [4] motion_1 sequence, this approach reduced the Absolute Trajectory Error (ATE) RMSE by approximately 10%, while on the rgbd.bonn.balloon [2] sequence, we observed an improvement of roughly 15%. These results indicate that masking dynamic objects improved trajectory accuracy in dynamic scenes.

Reconstruction example:

For evaluation we specifically picked datasets that contain moving objects/people. The sequence motion_1 is a scene where a person is moving around and sitting on a couch while the camera is capturing frames rotating around the

scene. An example frame from the scene can be seen in Figure 5.



Figure 5. Sample frame from motion_1 sequence showing person that causes fragments in reconstruction

Figure 6 shows the difference in reconstruction when using the SAM2 masks. As we can see, especially in the red circles, the image on the left contains significantly less fragments from the person that sat on the couch than the original DROID-SLAM reconstruction on the right image.

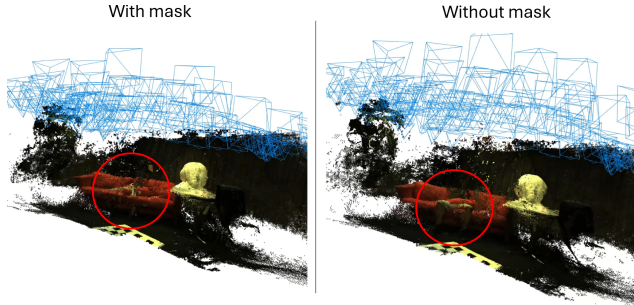


Figure 6. Comparison of the 3D reconstruction of our approach (using dynamic object masking with SAM2) to original DROID-SLAM without masks

4.1. User Study

To evaluate the usability of our system in a mixed reality setting, we conducted a user study on the Meta Quest headset. Participants were asked to interact with and select objects in the scene using their hands. The goal of the study was to assess the perceived ease, accuracy, and overall experience of object selection. Participants then completed a questionnaire giving their rating for different questions on a 5-point Likert scale.

We made participants aware of our GPU limitations and resulting mask latency that is visible but still asked questions regarding the latency to see how much it impacts user experience.

Question	Mean
Single-object selection ↑	4.18
Multi-object selection ↑	4.09
Latency noticeable ↓	3.45
Latency impact on UX ↓	2.64
Selection correctness ↑	4.27
Reconstruction quality ↑	4.27
Overall experience ↑	4.18

Table 1. User study results for object selection in MR (Meta Quest). We report mean values over $N = 11$ participants. ↑ means that higher values are preferred and ↓ means lower values are preferred.

The user study results in Table 1 show that participants experienced the object selection system as intuitive and reliable overall. High mean scores for single-object selection (4.18) and multi-object selection (4.09) suggest that users were generally able to select objects accurately. We then showed reconstruction results and asked for their rating when comparing the results from our methodology using masks to the results from the original DROID-SLAM and achieved a mean score of 4.27. Latency due to computational limitations was rated as noticeable (3.45). However, its impact on user experience was rated lower (2.64). This suggests that while users were aware of some delay, it did not significantly hinder their interaction.

Overall, these results show that the system provides a positive user experience that can be further improved by reducing latency.

5. Discussion & Limitations

In this work, we presented a client-server framework that leverages the state-of-the-art SAM2 foundation model to perform live dynamic object masking on the Meta Quest 3, thereby improving the robustness of DROID-SLAM in dynamic environments.

Strengths: The primary strength of our approach lies in its powerful and versatile tracking capabilities. By integrating SAM2, the system can robustly segment arbitrary moving objects without the need for domain-specific training. The client-server architecture effectively offloads computation, keeping the headset lightweight and ensuring that the visualization interface remains usable. Most importantly, our evaluation demonstrates that suppressing these segmented regions during the SLAM optimization process significantly improves trajectory estimation and reduces reconstruction artifacts in scenes populated by dynamic actors.

Limitations: The current system is heavily constrained by computational resources. Running the SAM2 model, particularly the large variant, requires significant GPU expenditure. As noted in the server methodology, VRAM requirements scale linearly with the number of objects (approx. 7GB per object), limiting the practical application to scenes with few dynamic elements. Additionally, the inference time on consumer-grade hardware (RTX 3090) leads to high latency and inconsistent framerates. Consequently, while the visualization is interactive, the final SLAM reconstruction must currently be processed offline using the recorded masks and video feed, rather than running simultaneously in real-time.

Another limitation is that the Meta Quest 3 is primarily designed as a VR headset and therefore not ideal for AR tasks that require precise alignment with the real world. First, the passthrough camera feed that is accessible through the Android Camera2 API has a significantly smaller FOV than the user’s actual visual field in passthrough mode. As illustrated in Figure 7, this mismatch restricts both user interaction and segmentation coverage to a central region of the view, limiting the selection of objects near the periphery. Second, accessing the camera feed via the Camera2 API introduces an additional latency of approximately 60ms. This delay compounds the already high inference latency of the segmentation model, further reducing temporal alignment between the real scene and the rendered segmentation overlay.

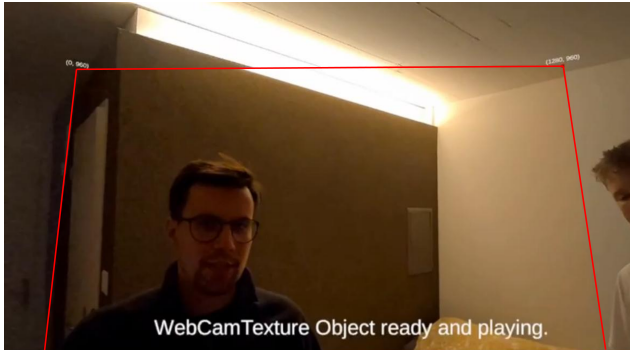


Figure 7. Only limited FOV of passthrough camera can be accessed and used for segmentation (highlighted with red bounds)

6. Conclusion & Future work:

Despite current hardware limitations, our approach validates that foundation models like SAM2 can be deployed effectively to help with the dynamic SLAM problem without specialized architectural changes. While the latency currently poses a challenge for fully synchronous online processing, the significant improvement in reconstruction

accuracy highlights the potential of this method as hardware capabilities continue to advance.

Recent advances in foundation models for visual segmentation offer promising directions for extending this work. With the recent release of SAM3 [1], which enables prompt based and object specific segmentation, future systems could be extended to automatically identify and select dynamic objects in the scene without requiring explicit user interaction. This would allow dynamic regions to be detected and suppressed in the SLAM optimization pipeline fully autonomously, further improving robustness in complex environments.

Beyond mixed reality applications, the proposed integration of dynamic object segmentation and SLAM has strong potential for real-world robotic and autonomous driving as these systems operate in highly dynamic environments.

References

- [1] Nicolas Carion, Laura Gustafson, Yuan-Ting Hu, Shoubhik Debnath, Ronghang Hu, Didac Suris, Chaitanya Ryali, Kalyan Vasudev Alwala, Haitham Khedr, Andrew Huang, et al. Sam 3: Segment anything with concepts. *arXiv preprint arXiv:2511.16719*, 2025. 6
- [2] Emanuele Palazzolo, Jens Behley, Philipp Lottes, Philippe Giguere, and Cyrill Stachniss. Refusion: 3d reconstruction in dynamic environments for rgb-d cameras exploiting residuals. In *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 7855–7862. IEEE, 2019. 1, 4
- [3] Nikhila Ravi, Valentin Gabeur, Yuan-Ting Hu, Ronghang Hu, Chaitanya Ryali, Tengyu Ma, Haitham Khedr, Roman Rädle, Chloe Rolland, Laura Gustafson, et al. Sam 2: Segment anything in images and videos. *arXiv preprint arXiv:2408.00714*, 2024. 1, 2, 3, 4
- [4] Thomas Schops, Torsten Sattler, and Marc Pollefeys. Bad slam: Bundle adjusted direct rgb-d slam. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 134–144, 2019. 1, 4
- [5] Zachary Teed and Jia Deng. Droid-slam: Deep visual slam for monocular, stereo, and rgb-d cameras. *Advances in neural information processing systems*, 34:16558–16569, 2021. 1, 2, 4
- [6] Jianhao Zheng, Zihan Zhu, Valentin Bieri, Marc Pollefeys, Songyou Peng, and Iro Armeni. Wildgs-slam: Monocular gaussian splatting slam in dynamic environments. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 11461–11471, 2025. 1